

JAVA PROGRAMMING

Chapter 27: File Handling in Java

Personal Notes

Up to now your programs have only worked with in-memory data — variables, arrays, lists, maps. The moment the program ends, everything disappears. Real applications need to persist data, read configuration, process logs, import CSVs. That's where file handling comes in.

Java's file I/O has evolved a lot over the years. The old way uses classes like `FileReader` and `FileWriter`; the modern way uses `Files`, `Path`, and `Stream` from the NIO.2 API (Java 7+). This chapter covers both, but leans heavily on the modern approach — it's what real code uses today.

1. Why File I/O Matters

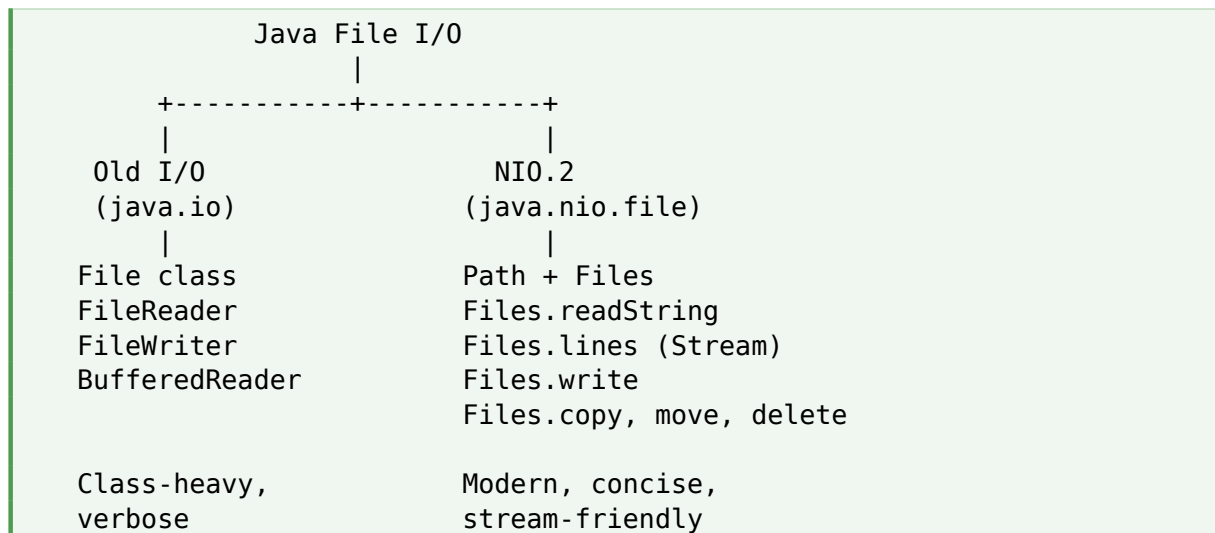
Files are how programs talk to the outside world for persistent data:

- Saving user data, settings, and game progress
- Reading configuration (`.properties`, `.json`, `.yaml` files)
- Processing CSVs, logs, reports
- Generating output documents, exports
- Reading and writing images, audio, video (binary I/O)

Almost every real-world Java application — backend, Android, desktop — touches files at some point. Getting comfortable here is non-negotiable.

2. The Big Picture

Java offers two main APIs for working with files:



Which to use: For new code, prefer the NIO.2 API (Path, Files). It's shorter, safer, and integrates with Streams. The older java.io classes still work, but they're verbose and require manual stream closing. You'll see both in real codebases — knowing both is essential.

3. Path — Representing a File Location

Path is an object that represents a file or directory location. It does NOT open the file — it just describes WHERE the file lives. Think of it as a smart filename.

Creating a Path

```
import java.nio.file.Path;
import java.nio.file.Paths;

// Modern way (Java 11+)
Path p1 = Path.of("data.txt");
Path p2 = Path.of("C:", "users", "akanshu", "notes.txt");

// Older way (Java 7+)
Path p3 = Paths.get("data.txt");
Path p4 = Paths.get("/home/akanshu/notes.txt");
```

Useful Path Methods

```
Path p = Path.of("/home/akanshu/notes/file.txt");

p.getFileName();      // "file.txt"
p.getParent();        // "/home/akanshu/notes"
p.getRoot();          // "/"
p.toString();         // "/home/akanshu/notes/file.txt"
p.toAbsolutePath();   // converts relative to absolute
p.isAbsolute();       // true / false
```

Important: A Path is just a name — creating one does NOT create the actual file. To do anything (read, write, check existence), you pass the Path to the Files class. The Path is the WHAT; Files provides the HOW.

4. Reading Files — The Modern Way

The Files class (java.nio.file.Files) provides the cleanest ways to read files. Pick the method that fits your data.

4.1 Read Entire File as a String

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
```

```
public class Main {
    public static void main(String[] args) throws IOException {
        String content = Files.readString(Path.of("data.txt"));
        System.out.println(content);
    }
}
```

One line, the whole file as a String. Perfect for small-to-medium files like configs.

4.2 Read All Lines into a List

```
List<String> lines = Files.readAllLines(Path.of("data.txt"));
for (String line : lines) {
    System.out.println(line);
}
```

Each line of the file becomes one entry in the list. Great when you want indexed access.

4.3 Stream Lines One at a Time (BEST for large files)

```
try (Stream<String> lines = Files.lines(Path.of("big.log"))) {
    lines.filter(line -> line.contains("ERROR"))
        .forEach(System.out::println);
}
```

Why streams matter for big files: `readString` and `readAllLines` load the WHOLE file into memory. For a 10 GB log file, that's a crash. `Files.lines()` reads line by line, lazily — you can process gigabytes with constant memory. Always wrap it in `try-with-resources` because the underlying file handle must be closed.

4.4 Read All Bytes (for Binary Files)

```
byte[] bytes = Files.readAllBytes(Path.of("image.png"));
System.out.println("File size: " + bytes.length + " bytes");
```

5. Reading Files — The Older Way

You'll still see this style in older codebases. It works, it's just more verbose.

5.1 BufferedReader (Line by Line)

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
```

```
        System.out.println(line);
    }
}
```

Read lines until `readLine()` returns null (end of file). The buffer makes it efficient for large files.

5.2 Scanner (Token-Based)

```
import java.util.Scanner;
import java.io.File;

try (Scanner sc = new Scanner(new File("data.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }

    // Or read numbers:
    while (sc.hasNextInt()) {
        int n = sc.nextInt();
        System.out.println(n);
    }
}
```

Scanner is friendly when you want to read by token (word, number, line). Slower than `BufferedReader` for line-by-line but more flexible for mixed input.

6. Writing Files — The Modern Way

6.1 Write a String

```
Files.writeString(Path.of("output.txt"), "Hello, file!");
// Overwrites if the file exists, creates if it doesn't
```

6.2 Write Multiple Lines

```
List<String> lines = Arrays.asList("Line 1", "Line 2", "Line 3");
Files.write(Path.of("output.txt"), lines);
```

6.3 Append Instead of Overwrite

```
import java.nio.file.StandardOpenOption;

Files.writeString(
    Path.of("log.txt"),
    "New entry\n",
    StandardOpenOption.CREATE,
    StandardOpenOption.APPEND
);
```

6.4 Write Bytes (for Binary Data)

```
byte[] data = ...;  
Files.write(Path.of("output.bin"), data);
```

StandardOpenOption — **useful flags:** CREATE — create if file doesn't exist (default behavior of write). CREATE_NEW — create only if file doesn't already exist (fails otherwise). APPEND — add to end of file. TRUNCATE_EXISTING — clear file before writing (default for write). WRITE — open for writing.

7. Writing Files — The Older Way

7.1 BufferedWriter

```
import java.io.BufferedWriter;  
import java.io.FileWriter;  
  
try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt")))  
{  
    bw.write("Hello");  
    bw.newLine();  
    bw.write("World");  
}  
  
// Append mode — pass true as second argument to FileWriter  
try (BufferedWriter bw = new BufferedWriter(new FileWriter("log.txt",  
true))) {  
    bw.write("New log entry");  
    bw.newLine();  
}
```

7.2 PrintWriter (Like System.out.println for Files)

```
import java.io.PrintWriter;  
import java.io.FileWriter;  
  
try (PrintWriter pw = new PrintWriter(new FileWriter("output.txt"))) {  
    pw.println("Hello");  
    pw.printf("Pi is approximately %.2f%n", 3.14159);  
    pw.write("Direct text");  
}
```

PrintWriter is the most beginner-friendly — println, printf, write all work just like with System.out.

8. File Operations

The Files class also provides utility methods for creating, deleting, moving, and checking files.

Method	What it does
Files.exists(path)	Does the file/folder exist?
Files.notExists(path)	Confirm it doesn't exist
Files.isRegularFile(path)	Is it a regular file (not a directory)?
Files.isDirectory(path)	Is it a directory?
Files.isReadable(path)	Can we read it?
Files.isWritable(path)	Can we write to it?
Files.size(path)	File size in bytes
Files.createFile(path)	Create an empty file
Files.createDirectory(path)	Create a single directory
Files.createDirectories(path)	Create directory + any missing parents
Files.delete(path)	Delete — throws if missing
Files.deleteIfExists(path)	Delete — silent if missing
Files.copy(src, dest)	Copy a file
Files.move(src, dest)	Move/rename a file

Example: Safe File Creation

```

Path file = Path.of("data.txt");

if (!Files.exists(file)) {
    Files.createFile(file);
    System.out.println("Created: " + file);
} else {
    System.out.println("Already exists: " + file);
    System.out.println("Size: " + Files.size(file) + " bytes");
}

```

9. try-with-resources — Always Use It

Anything that opens a file produces a RESOURCE that must be closed. Forgetting to close leaks file handles — and on some systems, prevents other programs from accessing the file. Java's try-with-resources handles this automatically.

BAD — Manual Closing (Old Style)

```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("data.txt"));
    String line = br.readLine();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (br != null) {
        try { br.close(); } catch (IOException e) { /* swallow */ }
    }
}
```

GOOD — try-with-resources

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line = br.readLine();
} catch (IOException e) {
    e.printStackTrace();
}
// br.close() called automatically, even if an exception is thrown
```

9.1 Multiple Resources in One Try

```
try (
    BufferedReader in = Files.newBufferedReader(Path.of("input.txt"));
    BufferedWriter out = Files.newBufferedWriter(Path.of("output.txt"))
) {
    String line;
    while ((line = in.readLine()) != null) {
        out.write(line.toUpperCase());
        out.newLine();
    }
}
// Both resources closed automatically, in reverse order
```

10. Real-World Examples

10.1 Read a CSV File

```
// CSV file: name,age,city
```

```
try (Stream<String> lines = Files.lines(Path.of("users.csv"))) {
    lines.skip(1) // skip header
        .map(line -> line.split(","))
        .forEach(fields -> System.out.println(
            "Name: " + fields[0] +
            ", Age: " + fields[1] +
            ", City: " + fields[2]
        ));
}
```

10.2 Count Words in a File

```
long wordCount = 0;
try (Stream<String> lines = Files.lines(Path.of("article.txt"))) {
    wordCount = lines
        .flatMap(line -> Arrays.stream(line.split("\\s+")))
        .filter(w -> !w.isEmpty())
        .count();
}
System.out.println("Total words: " + wordCount);
```

10.3 Filter ERROR Lines from a Log

```
Path input = Path.of("app.log");
Path output = Path.of("errors.log");

try (
    Stream<String> lines = Files.lines(input);
    BufferedWriter bw    = Files.newBufferedWriter(output)
) {
    lines.filter(line -> line.contains("ERROR"))
        .forEach(line -> {
            try {
                bw.write(line);
                bw.newLine();
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
}
```

10.4 Append a Log Entry

```
String entry = LocalDateTime.now() + " - User logged in\n";
Files.writeString(
    Path.of("activity.log"),
    entry,
    StandardOpenOption.CREATE,
    StandardOpenOption.APPEND
);
```



```
);
```

10.5 Copy and Rename Files

```
Path source = Path.of("data.txt");
Path backup = Path.of("data_backup.txt");

// Copy (overwrites if backup already exists)
Files.copy(source, backup, StandardCopyOption.REPLACE_EXISTING);

// Rename
Files.move(Path.of("old.txt"), Path.of("new.txt"));
```

11. Common Exceptions

Exception	When it happens
IOException	General I/O failure — must be caught (checked)
FileNotFoundException	Trying to read a file that doesn't exist
NoSuchFileException	NIO version — Files.delete() on missing file
AccessDeniedException	No permission to read or write
DirectoryNotEmptyException	Trying to delete a non-empty directory
FileAlreadyExistsException	CREATE_NEW when the file already exists

All these are checked exceptions — the compiler will force you to either catch them or declare them with throws.

12. Common Mistakes

Mistake 1: Forgetting to close files

Without try-with-resources, it's very easy to forget close() — especially when exceptions interrupt the flow. Unclosed files leak handles. Always use try-with-resources.

Mistake 2: Loading huge files entirely

```
// BAD for big files – loads everything into memory
List<String> lines = Files.readAllLines(Path.of("10gb_log.txt"));

// GOOD – streams line by line, constant memory
try (Stream<String> lines = Files.lines(Path.of("10gb_log.txt"))) {
```

```
    lines.forEach(this::process);  
}
```

Mistake 3: Using == to compare Paths

```
Path a = Path.of("data.txt");  
Path b = Path.of("data.txt");  
  
a == b           // FALSE – different objects  
a.equals(b)      // TRUE – same content
```

Mistake 4: Hard-coding OS-specific separators

```
Path p = Path.of("data\\file.txt");    // Windows-only  
Path p = Path.of("data/file.txt");     // Unix-only  
  
// Better – let Path handle separators:  
Path p = Path.of("data", "file.txt");
```

Mistake 5: Ignoring character encoding

```
// May break with non-ASCII files on some systems  
Files.readString(Path.of("data.txt"));  
  
// Safer – explicit UTF-8  
Files.readString(Path.of("data.txt"), StandardCharsets.UTF_8);
```

Mistake 6: Catching IOException too broadly

Many bugs hide behind a catch (IOException e) {} that silently swallows errors. At minimum, print the stack trace or log the message. Don't ignore I/O errors.

Mistake 7: Using getAbsolutePath() vs toAbsolutePath()

File.getAbsolutePath() returns a String. Path.toAbsolutePath() returns a Path. Don't mix them up — they look similar but the return types differ.

13. Best Practices

- Default to the NIO.2 API (Files, Path) for new code
- Always use try-with-resources — never close files manually
- For large files, use Files.lines() with streams, not readAllLines
- Specify encoding explicitly (StandardCharsets.UTF_8) for text files
- Use Path.of("a", "b") to avoid platform-specific separators
- Check Files.exists() before reading; check Files.isWritable() before writing
- Catch IOException specifically — and at minimum, log it
- For appending logs, use APPEND with CREATE to handle both new and existing files

- Don't build huge in-memory lists from files when you only need to process them

14. Quick Reference

Method / Class	Meaning
Path.of("file.txt")	Create a Path from a string
Files.exists(path)	Does the file exist?
Files.readString(path)	Read entire file as a String
Files.readAllLines(path)	Read all lines as List<String>
Files.lines(path)	Stream lines (lazy, big-file safe)
Files.readAllBytes(path)	Read entire file as byte[]
Files.writeString(path, s)	Write a String to a file
Files.write(path, lines)	Write lines
Files.write(path, bytes)	Write raw bytes
Files.newBufferedReader(p)	BufferedReader for line-by-line
Files.newBufferedWriter(p)	BufferedWriter for writing
Files.copy(src, dest)	Copy a file
Files.move(src, dest)	Move/rename a file
Files.delete(path)	Delete a file
Files.createFile(path)	Create an empty file
Files.createDirectories(path)	Create nested directories
StandardOpenOption.APPEND	Flag for append mode
StandardCharsets.UTF_8	Explicit UTF-8 encoding

15. Practice Problems

Try each one — every problem teaches a real-world file pattern.

1. Read a text file and print its contents using `Files.readString`.
2. Read a file line by line and print each line with its number (Line 1: ..., Line 2: ...).
3. Write "Hello, Java!" to a file. Then read it back to confirm.
4. Append a new log entry with current timestamp to `log.txt`.
5. Read a CSV file with name,age,city columns. Print each name.
6. Count the number of lines in a file.
7. Count occurrences of the word "Java" in a file.
8. Copy contents of `input.txt` to `output.txt`, converting all text to uppercase.
9. Filter only the ERROR lines from a log file into a new `errors.log`.
10. Build a word frequency map from a file using streams + `Collectors.groupingBy`.
11. Create a directory structure: `data/2026/reports/`, then write a file inside it.
12. Delete all `.tmp` files in a directory (use `Files.list` and filter by extension).
13. Read numbers (one per line) from a file and print their sum and average.
14. Take input from the user via `Scanner` and write it to a file (line-by-line until they type "exit").

16. Final Takeaway

File handling ties together everything you've learned — exceptions (Chapter 23), try-with-resources, streams (Chapter 25), lambdas. A clean file-processing pipeline in modern Java is just streams flowing from a `Path` through transformations to either an output file or a result.

Master the NIO.2 API (`Files` + `Path`), use try-with-resources religiously, and lean on `Files.lines()` for any file bigger than "small." That's 90% of practical file I/O — and the foundation for working with databases, APIs, and any external data.

Key Takeaway: `Files.readString` for small text. `Files.readAllLines` when you need a `List`. `Files.lines` for big files. `Files.write` for output. Always try-with-resources. Always specify UTF-8 for text. Those 5 rules cover most real-world file work — and the rest is just file system commands (copy, move, delete) that follow the same pattern.

17. What's Next?

Now that file I/O is solid, the natural next topics are:

- Multithreading — `Thread`, `Runnable`, `ExecutorService` for parallel work
- Concurrency utilities — `synchronized`, locks, `AtomicInteger`, `ConcurrentHashMap`
- Date and Time API — `java.time`, `LocalDate`, `Instant`, `Duration`
- Regular expressions — `Pattern` and `Matcher` for text matching
- Serialization — saving objects to disk with `Serializable`
- Working with JSON — `Jackson`, `Gson`

- Networking — Sockets, HttpClient (Java 11+)
- Java 9+ features — var, modules, switch expressions, text blocks